Lab 09 – Worksheet

Name: Myrah Hussain Romaisa Shazad	ID: mh10074 rb10107	Section:

Note: Assumptions and logics should be explained separately in tasks after the task results.

Github link:
https://github.com/myrahhussain/CA_Labs_MyrahHussain_RomaisaShazad

Task 1

Truth Table
a) Main Control Unit

Instruction	Opcode	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	ALUOp
R-type (ADD,SUB,SLL,SRL,AND,OR,XOR)	0110011	1	0	0	0	0	0	10
I-type ALU (ADDI)	0010011	1	1	0	0	0	0	11
Load (LW,LH,LB)	0000011	1	1	1	0	1	0	00
Store (SW,SH,SB)	0100011	0	1	0	1	X	0	00
Branch (BEQ)	1100011	0	0	0	0	X	1	01

b) ALU Control Unit

ALU control unit (ALUOp, funct3, funct7 as inputs)

ALUOp	funct3	funct7[5]	Operation	ALUControl
00	XXX	X	ADD (load/store)	0010
01	XXX	X	SUB (branch compare)	0110
10	000	0	ADD	0010
10	000	1	SUB	0110
10	001	X	SLL	0011
10	101	X	SRL	0000
10	110	X	OR	0001
10	111	X	AND	1000
10	100	X	XOR	0111
11	000	X	ADDI	0010
11	001	X	SLLI	0011
11	101	X	SRLI	0000
11	110	X	ORI	0001
11	111	X	ANDI	1000
11	100	X	XORI	0111

Comments

Main Control Unit Table:

The Main Control Unit table maps the opcode to all datapath control signals. The opcode alone is enough to determine RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch, and ALUOp for each instruction type since all instructions of the same type (e.g. all loads) need identical control signals regardless of funct3.

R-type: Both operands from registers (ALUSrc=0), result written to register (RegWrite=1), no memory, ALUOp=10 to pass control to ALU unit.

I-type ALU: Same as R-type but ALUSrc=1 for immediate operand, ALUOp=11 to distinguish from loads.

Load: ALUSrc=1 for address calculation, MemRead=1 and MemtoReg=1 to fetch and write loaded value, RegWrite=1, ALUOp=00 forces ADD.

Store: MemWrite=1, RegWrite=0, MemtoReg is don't-care, ALUOp=00 forces ADD for address.

BEQ: Branch=1, ALUSrc=0, RegWrite=0, all memory signals 0, ALUOp=01 forces SUB for comparison.

ALU Control Unit Table:

ALU Control Unit Table takes ALUOp + funct3 + funct7[5] and produces the exact 4-bit ALUControl signal.

ALUOp=00 (Load/Store): funct3 and funct7 are don't-care, always outputs ADD since only address calculation is needed.

ALUOp=01 (Branch): Always outputs SUB regardless of funct fields, so the zero flag can evaluate the branch condition.

ALUOp=10 (R-type) and ALUOp=11 (I-type): funct3 selects the operation. The only special case is funct3=000 where funct7[5] differentiates ADD (0) from SUB (1). For I-type funct7 is don't-care at funct3=000 since SUBI does not exist in RISC-V.

Task 2

*Design Module Codes for **Main Control Unit,** and **ALU Control Unit.***

1. **Main Control Unit:**

- The module takes a 7-bit opcode as input and outputs all control signals needed by the datapath: RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch, and ALUOp[1:0]
- Instead of writing raw binary opcodes in the case statement, named constants (R_TYPE, I_ALU, LOAD, STORE, BRANCH) are defined using localparam to make the code more readable
- The always block uses always @(*), making it combinational logic. It re-evaluates whenever any input changes. At the start, all outputs are defaulted to 0 to avoid unintended latches and handle unknown opcodes safely
- For **R_TYPE**, RegWrite=1 to write result back, ALUSrc=0 since both operands come from registers, ALUOp=10 to tell ALU control to look at funct3/funct7
- For **I_ALU**, same as R-type but ALUSrc=1 since second operand is an immediate, ALUOp=11 to distinguish it from loads
- For **LOAD**, ALUSrc=1 and ALUOp=00 for address calculation (always ADD), MemRead=1 to read from memory, MemtoReg=1 to route loaded data to the register file
- For **STORE**, MemWrite=1 and RegWrite=0 since nothing is written to a register, ALUOp=00 for address calculation
- For **BRANCH**, Branch=1, ALUSrc=0 since both operands are registers, ALUOp=01 to force SUB for equality check
- The default case de-asserts all signals if an unrecognized opcode is received, preventing any unintended memory writes or register updates

```
`timescale 1ns / 1ps
// ==========================================================
// MainControl.v
// Main Control Unit for Single-Cycle RISC-V (RV32I)
// Inputs:  opcode[6:0]
// Outputs: RegWrite, ALUSrc, MemRead, MemWrite,
//          MemtoReg, Branch, ALUOp[1:0]
// ==========================================================

module MainControl (
   input  wire [6:0] opcode,
   output reg       RegWrite,
```

```
    output reg        ALUSrc,
    output reg        MemRead,
    output reg        MemWrite,
    output reg        MemtoReg,
    output reg        Branch,
    output reg [1:0]  ALUOp
);

    // RISC-V opcode definitions
    localparam R_TYPE  = 7'b0110011;  // ADD, SUB, SLL, SRL, AND, OR,
XOR
    localparam I_ALU   = 7'b0010011;  // ADDI, ANDI, ORI, XORI, SLLI, SRLI
    localparam LOAD    = 7'b0000011;  // LW, LH, LB
    localparam STORE   = 7'b0100011;  // SW, SH, SB
    localparam BRANCH  = 7'b1100011;  // BEQ

    always @(*) begin
        // Safe default values (all signals de-asserted)
        RegWrite = 1'b0;
        ALUSrc   = 1'b0;
        MemRead  = 1'b0;
        MemWrite = 1'b0;
        MemtoReg = 1'b0;
        Branch   = 1'b0;
        ALUOp    = 2'b00;

        case (opcode)
            // ------------------------------------------------
            // R-type: ADD, SUB, SLL, SRL, AND, OR, XOR
            // ALU operation determined by funct3/funct7 (ALUOp=10)
            // ------------------------------------------------
            R_TYPE: begin
                RegWrite = 1'b1;
                ALUSrc   = 1'b0;  // second ALU operand from register
                MemRead  = 1'b0;
                MemWrite = 1'b0;
                MemtoReg = 1'b0;  // write ALU result to register
                Branch   = 1'b0;
                ALUOp    = 2'b10;
```

```
end

// ------------------------------------------------
// I-type ALU: ADDI (and other immediate operations)
// ALUOp=11 lets ALU control differentiate from loads
// ------------------------------------------------
I_ALU: begin
   RegWrite = 1'b1;
   ALUSrc   = 1'b1;  // second ALU operand from immediate
   MemRead  = 1'b0;
   MemWrite = 1'b0;
   MemtoReg = 1'b0;  // write ALU result to register
   Branch   = 1'b0;
   ALUOp    = 2'b11;
end

// ------------------------------------------------
// Load: LW, LH, LB
// ALU computes address (add), ALUOp=00
// ------------------------------------------------
LOAD: begin
   RegWrite = 1'b1;
   ALUSrc   = 1'b1;  // immediate as offset
   MemRead  = 1'b1;
   MemWrite = 1'b0;
   MemtoReg = 1'b1;  // write memory data to register
   Branch   = 1'b0;
   ALUOp    = 2'b00;
end

// ------------------------------------------------
// Store: SW, SH, SB
// No register write; ALU computes address (add), ALUOp=00
// ------------------------------------------------
STORE: begin
   RegWrite = 1'b0;
   ALUSrc   = 1'b1;  // immediate as offset
   MemRead  = 1'b0;
   MemWrite = 1'b1;
```

```
            MemtoReg = 1'b0;  // don't care - no reg write
            Branch   = 1'b0;
            ALUOp    = 2'b00;
        end

        // ------------------------------------------------
        // Branch: BEQ
        // ALU subtracts rs1-rs2 to test equality, ALUOp=01
        // ------------------------------------------------
        BRANCH: begin
            RegWrite = 1'b0;
            ALUSrc   = 1'b0;  // both operands from registers
            MemRead  = 1'b0;
            MemWrite = 1'b0;
            MemtoReg = 1'b0;  // don't care - no reg write
            Branch   = 1'b1;
            ALUOp    = 2'b01;
        end

        // ------------------------------------------------
        // Default: unknown opcode - de-assert all signals
        // ------------------------------------------------
        default: begin
            RegWrite = 1'b0;
            ALUSrc   = 1'b0;
            MemRead  = 1'b0;
            MemWrite = 1'b0;
            MemtoReg = 1'b0;
            Branch   = 1'b0;
            ALUOp    = 2'b00;
        end
    endcase
end

endmodule
```

2. **ALU Control module:**

- The module takes ALUOp[1:0] from the Main Control, funct3[2:0] and funct7[6:0] from the instruction, and outputs a 4-bit ALUControl signal that tells the ALU which operation to perform
- Only funct7[5] (instruction bit [30]) is extracted and used; this is the only bit needed to distinguish ADD from SUB when funct3=000
- The ALUControl encoding is defined in the comments: 0000=SRL, 0001=OR, 0010=ADD, 0011=SLL, 0110=SUB, 0111=XOR, 1000=AND
- casez is used instead of a regular case statement because it supports don't-care matching using ?, which is needed since funct3 and funct7[5] are irrelevant for ALUOp=00 and ALUOp=01
- All six inputs (ALUOp, funct3, f7b5) are concatenated into a 6-bit value {ALUOp, funct3, f7b5} so they can be matched in a single case statement
- For **ALUOp=00** (Load/Store), funct3 and funct7 are don't-care, always outputs ADD for address calculation
- For **ALUOp=01** (Branch), same don't-care approach, always outputs SUB so the zero flag can evaluate the branch condition
- For **ALUOp=10** (R-type), funct3 selects the operation. The only special case is funct3=000 where funct7[5]=0 gives ADD and funct7[5]=1 gives SUB. All other funct3 values have funct7[5] as don't-care
- For **ALUOp=11** (I-type), funct7[5] is don't-care for all entries since funct7 bits are part of the immediate value in I-type instructions, not a function code
- The default case outputs ADD to ensure a safe known value if no pattern is matched

```
`timescale 1ns / 1ps
// ==========================================================
// ALUControl.v
// ALU Control Unit for Single-Cycle RISC-V (RV32I)
//
// Inputs:  ALUOp[1:0]  - from Main Control
//          funct3[2:0] - instruction bits [14:12]
//          funct7[6:0] - instruction bits [31:25]
//                        only funct7[5] (instr bit [30])
//                        is used to distinguish ADD/SUB
// Output:  ALUControl[3:0]
```

```
//
// ALUControl encoding:
//   0000 = SRL
//   0001 = OR
//   0010 = ADD
//   0011 = SLL
//   0110 = SUB
//   0111 = XOR
//   1000 = AND
// ============================================================

module ALUControl (
   input  wire [1:0] ALUOp,
   input  wire [2:0] funct3,
   input  wire [6:0] funct7,   // full funct7, bits [31:25] of instruction
   output reg  [3:0] ALUControl
);

   // funct7[5] corresponds to instruction bit [30]
   // it is the only bit that distinguishes ADD (0) from SUB (1)
   wire f7b5 = funct7[5];

   always @(*) begin
      // Safe default
      ALUControl = 4'b0010; // ADD

      casez ({ALUOp, funct3, f7b5})
         // ------------------------------------------------
         // ALUOp = 00 : Load / Store ? always ADD
         // ------------------------------------------------
         6'b00_???_? : ALUControl = 4'b0010; // ADD

         // ------------------------------------------------
         // ALUOp = 01 : Branch (BEQ) ? always SUB
         // ------------------------------------------------
         6'b01_???_? : ALUControl = 4'b0110; // SUB

         // ------------------------------------------------
         // ALUOp = 10 : R-type
```

```
        // funct7[5]=0 ? ADD, funct7[5]=1 ? SUB
        // ------------------------------------------------
        6'b10_000_0 : ALUControl = 4'b0010; // ADD
        6'b10_000_1 : ALUControl = 4'b0110; // SUB
        6'b10_001_? : ALUControl = 4'b0011; // SLL
        6'b10_101_? : ALUControl = 4'b0000; // SRL
        6'b10_110_? : ALUControl = 4'b0001; // OR
        6'b10_111_? : ALUControl = 4'b1000; // AND
        6'b10_100_? : ALUControl = 4'b0111; // XOR

        // ------------------------------------------------
        // ALUOp = 11 : I-type ALU
        // funct7 is part of the immediate - ignore funct7[5]
        // ------------------------------------------------
        6'b11_000_? : ALUControl = 4'b0010; // ADDI
        6'b11_001_? : ALUControl = 4'b0011; // SLLI
        6'b11_101_? : ALUControl = 4'b0000; // SRLI
        6'b11_110_? : ALUControl = 4'b0001; // ORI
        6'b11_111_? : ALUControl = 4'b1000; // ANDI
        6'b11_100_? : ALUControl = 4'b0111; // XORI

        default     : ALUControl = 4'b0010; // ADD
      endcase
    end

endmodule
```

Testbench:

- This is the testbench that verifies all 15 supported RISC-V instructions by instantiating both MainControl and ALUControl together and checking their outputs against expected values
- The inputs (opcode, funct3, funct7) are declared as `reg` since they are driven by the testbench, and all control signal outputs are declared as `wire` since they come from the modules under test

- Both modules are instantiated and connected directly — ALUOp output from MainControl feeds straight into ALUControl, mimicking the actual hardware connection
- The `apply_and_check` task is defined to avoid repeating the same code 15 times. It takes the instruction name, input values, and all expected outputs as arguments, applies the inputs, waits 10ns, then compares each output against its expected value and prints either [PASS] or [FAIL] with details
- All inputs are initialized to 0 at the start, with a 5ns delay before any instruction is applied. This prevents X (unknown) states from appearing at t=0
- The instructions are tested in groups — R-type first (ADD through XOR), then I-type (ADDI), then Loads (LW/LH/LB), then Stores (SW/SH/SB), and finally BEQ
- For R-type, funct7=0000000 is used for all operations except SUB which uses funct7=0100000 (bit[5]=1) to correctly trigger the SUB case in ALUControl
- For Load, Store, Branch, and I-type, funct7 is always set to 0000000 since it is either part of the immediate or not used at all by the control logic

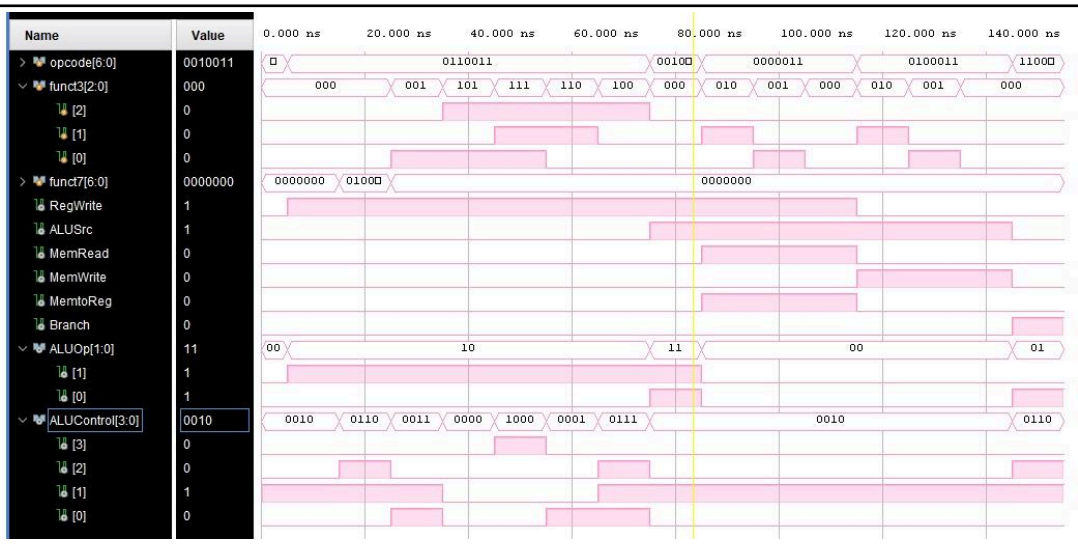
```
`timescale 1ns / 1ps
// ==========================================================
// tb_lab09.v
// Testbench for MainControl + ALUControl
// funct7 is full 7 bits [31:25] of the instruction
// only funct7[5] (instr bit [30]) distinguishes ADD/SUB
// ==========================================================

module tb_lab09;

   reg  [6:0] opcode;
   reg  [2:0] funct3;
   reg  [6:0] funct7;   // full 7-bit funct7 [31:25]

   wire       RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch;
   wire [1:0] ALUOp;
   wire [3:0] ALUControl;

   MainControl uut_main (
      .opcode   (opcode),
      .RegWrite (RegWrite),
      .ALUSrc   (ALUSrc),
      .MemRead  (MemRead),
```

```
    .MemWrite (MemWrite),
    .MemtoReg (MemtoReg),
    .Branch   (Branch),
    .ALUOp    (ALUOp)
);

ALUControl uut_alu (
    .ALUOp      (ALUOp),
    .funct3     (funct3),
    .funct7     (funct7),
    .ALUControl (ALUControl)
);

task apply_and_check;
    input [63:0] instr_name;
    input [6:0]  op;
    input [2:0]  f3;
    input [6:0]  f7;   // full 7-bit funct7
    input        exp_RegWrite, exp_ALUSrc, exp_MemRead, exp_MemWrite;
    input        exp_MemtoReg, exp_Branch;
    input [1:0]  exp_ALUOp;
    input [3:0]  exp_ALUCtrl;
    begin
        opcode = op;
        funct3 = f3;
        funct7 = f7;
        #10;

        $display("-----------------------------------------------");
        $display(" Instruction : %s", instr_name);
        $display(" opcode=%b  funct3=%b  funct7=%b", opcode, funct3, funct7);
        $display(" RegWrite=%b ALUSrc=%b MemRead=%b MemWrite=%b
MemtoReg=%b Branch=%b",
                 RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch);
        $display(" ALUOp=%b  ALUControl=%b", ALUOp, ALUControl);

        if (RegWrite   !== exp_RegWrite)  $display("  [FAIL] RegWrite  : got %b,
exp %b", RegWrite,   exp_RegWrite);
        if (ALUSrc     !== exp_ALUSrc)    $display("  [FAIL] ALUSrc    : got %b,
exp %b", ALUSrc,     exp_ALUSrc);
        if (MemRead    !== exp_MemRead)   $display("  [FAIL] MemRead   : got
%b, exp %b", MemRead,    exp_MemRead);
```

```
        if (MemWrite   !== exp_MemWrite)  $display("  [FAIL] MemWrite  : got
%b, exp %b", MemWrite,   exp_MemWrite);
        if (Branch     !== exp_Branch)    $display("  [FAIL] Branch    : got %b, exp
%b", Branch,     exp_Branch);
        if (ALUOp      !== exp_ALUOp)     $display("  [FAIL] ALUOp     : got %b,
exp %b", ALUOp,      exp_ALUOp);
        if (ALUControl !== exp_ALUCtrl)   $display("  [FAIL] ALUControl: got
%b, exp %b", ALUControl, exp_ALUCtrl);
        else $display("  [PASS]");
      end
   endtask

   initial begin
      $dumpfile("tb_lab09.vcd");
      $dumpvars(0, tb_lab09);

      // Initialize all inputs to avoid X state at t=0
      opcode = 7'b0;
      funct3 = 3'b0;
      funct7 = 7'b0;
      #5;

      $display("=== RISC-V Control Path Testbench ===");

      // ------------------------------------------------
      // R-type (opcode=0110011)
      // funct7=0000000 for ADD,SLL,SRL,AND,OR,XOR
      // funct7=0100000 for SUB (bit[5]=1)
      // ------------------------------------------------
      apply_and_check("ADD ",  7'b0110011, 3'b000, 7'b0000000,  1,0,0,0, 0,0,
2'b10, 4'b0010);
      apply_and_check("SUB ",  7'b0110011, 3'b000, 7'b0100000,  1,0,0,0, 0,0,
2'b10, 4'b0110);
      apply_and_check("SLL ",  7'b0110011, 3'b001, 7'b0000000,  1,0,0,0, 0,0,
2'b10, 4'b0011);
      apply_and_check("SRL ",  7'b0110011, 3'b101, 7'b0000000,  1,0,0,0, 0,0,
2'b10, 4'b0000);
      apply_and_check("AND ",  7'b0110011, 3'b111, 7'b0000000,  1,0,0,0, 0,0,
2'b10, 4'b1000);
      apply_and_check("OR  ",  7'b0110011, 3'b110, 7'b0000000,  1,0,0,0, 0,0,
2'b10, 4'b0001);
      apply_and_check("XOR ",  7'b0110011, 3'b100, 7'b0000000,  1,0,0,0, 0,0,
2'b10, 4'b0111);
```

```
    // ---------------------------------------------------
    // I-type ALU (opcode=0010011)
    // funct7 is part of the immediate, use 0000000
    // ---------------------------------------------------
    apply_and_check("ADDI",  7'b0010011, 3'b000, 7'b0000000,  1,1,0,0, 0,0,
2'b11, 4'b0010);

    // ---------------------------------------------------
    // Load (opcode=0000011)
    // funct7 not used, always 0000000
    // ---------------------------------------------------
    apply_and_check("LW  ",  7'b0000011, 3'b010, 7'b0000000,  1,1,1,0, 1,0,
2'b00, 4'b0010);
    apply_and_check("LH  ",  7'b0000011, 3'b001, 7'b0000000,  1,1,1,0, 1,0,
2'b00, 4'b0010);
    apply_and_check("LB  ",  7'b0000011, 3'b000, 7'b0000000,  1,1,1,0, 1,0,
2'b00, 4'b0010);

    // ---------------------------------------------------
    // Store (opcode=0100011)
    // funct7 not used, always 0000000
    // ---------------------------------------------------
    apply_and_check("SW  ",  7'b0100011, 3'b010, 7'b0000000,  0,1,0,1, 0,0,
2'b00, 4'b0010);
    apply_and_check("SH  ",  7'b0100011, 3'b001, 7'b0000000,  0,1,0,1, 0,0,
2'b00, 4'b0010);
    apply_and_check("SB  ",  7'b0100011, 3'b000, 7'b0000000,  0,1,0,1, 0,0,
2'b00, 4'b0010);

    // ---------------------------------------------------
    // Branch (opcode=1100011)
    // funct7 not used, always 0000000
    // ---------------------------------------------------
    apply_and_check("BEQ ",  7'b1100011, 3'b000, 7'b0000000,  0,0,0,0, 0,1,
2'b01, 4'b0110);

    $display("=== Testbench complete ===");
    $finish;
  end

endmodule
```

Waveforms:



Simulation Waveform Explanation

The testbench applies all 15 instructions sequentially, each held for 10ns. Two waveform views are used, one in hex/decimal for quick pattern recognition and one in binary for bit-level verification.

R-type Instructions (opcode = 0110011)

The first opcode block covers all seven R-type instructions: ADD, SUB, SLL, SRL, AND, OR, and XOR. Since all share the same opcode, the Main Control unit produces identical outputs for all seven — RegWrite = 1, ALUSrc = 0, MemRead = 0, MemWrite = 0, MemtoReg = 0, Branch = 0, and ALUOp = 10. This is correctly reflected in the waveform where these signals remain stable across the entire R-type block.

The distinguishing work is done by the ALU Control unit, which decodes funct3 and funct7[5] to produce the correct ALUControl signal for each instruction. The waveform shows ALUControl cycling through the values 0010 → 0110 → 0011 → 0000 → 1000 → 0001 → 0111, corresponding to ADD, SUB, SLL, SRL, AND, OR, and XOR, respectively. The ADD/SUB distinction is correctly resolved by funct7[5]: when funct7[5] = 0, the output is 0010 (ADD), and when funct7[5] = 1, the output is 0110 (SUB), confirming correct implementation of this special case.

I-type ALU Instructions (opcode = 0010011)

For ADDI (and other I-type ALU operations such as SLLI, SRLI, ORI, ANDI, XORI), the opcode changes to 0010011. The key difference from R-type visible in the waveform is that ALUSrc flips to 1, indicating the second ALU operand is now sourced from the **sign-extended immediate** rather than a register. ALUOp is set to 11 to signal the ALU Control that this is an I-type instruction, allowing it to correctly suppress the SUB case for funct3 = 000 (always ADDI, never SUBI). RegWrite remains 1, and memory and branch signals remain deasserted.

Load Instructions (opcode = 0000011)

The load block covers LW, LH, and LB, differentiated by funct3 values 010, 001, and 000 respectively. The waveform correctly shows MemRead = 1 and MemtoReg = 1 asserted together — these are the two signals that uniquely identify a load. ALUSrc = 1 is also asserted since the memory address is computed as rs1 + sign-extended immediate. ALUOp = 00 forces the ALU Control to always output ADD (0010) regardless of funct3, since address calculation always requires addition. RegWrite = 1 confirms the loaded value will be written back to the destination register. All three load variants produce identical control signals, which is expected since funct3

only affects the datapath (byte/halfword/word selection), not the control unit outputs.

Store Instructions (opcode = 0100011)

The store block covers SW, SH, and SB. The waveform shows MemWrite = 1 and RegWrite = 0 — the defining characteristic of a store, since data is written to memory and no register is updated. ALUSrc = 1 and ALUOp = 00 are asserted for the same reasons as loads (address = rs1 + immediate, always ADD). MemRead = 0 and Branch = 0 remain deasserted. MemtoReg is a don't-care since RegWrite = 0 means the MemtoReg mux output is never used. As with loads, all three store variants produce the same control signals.

Branch Instruction — BEQ (opcode = 1100011)

BEQ is the final and only instruction to assert Branch = 1, which is clearly visible as a rising edge at the end of the waveform. ALUSrc = 0 because both operands for the comparison come from registers. ALUOp = 01 instructs the ALU Control to unconditionally output SUB (0110), so the ALU subtracts the two operands and the zero flag determines whether the branch is taken. RegWrite = 0, MemRead = 0, and MemWrite = 0 are all deasserted since BEQ neither reads/writes memory nor writes to a register.

Task 3

Design module codes of ***top module:***

Top_Lab09:

- This is the top-level wrapper module that connects everything for FPGA implementation. It takes the clock, reset, 16 switches, and a push button as inputs, and drives 16 LEDs as outputs
- The push button is first passed through a debouncer module (reused from Lab 5) to remove any mechanical noise and produce a clean signal, btn_clean
- The switches module (also reused from Lab 5) reads the physical switch positions and outputs a 32-bit sw_readData. From this, opcode[6:0] is extracted from SW[15:9], funct3[2:0] from SW[8:6], and funct7[5] from SW[5]
- MainControl and ALUControl are instantiated and connected. MainControl takes the latched opcode and produces all datapath control signals, which are then fed into ALUControl along with funct3 and funct7 to produce ALUControl[3:0]
- The LEDs are mapped as follows: LED[15:10] shows the main control signals (RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch), LED[9:8] shows ALUOp, LED[7:4] shows ALUControl, and LED[3:2] shows the current FSM state
- A 4-state FSM is implemented to control the sampling and display flow. **IDLE (00)** waits for a button press. **SAMPLE (01)** latches the current switch values into registers. **DISPLAY (10)** maps all control signals to the LED register. **WAIT_REL (11)** waits for the button to be released before returning to IDLE
- Rising edge detection is done using btn_rise = btn_clean & ~btn_prev — this ensures the FSM only advances once per button press and not continuously while held

```
`timescale 1ns / 1ps
// ============================================================
// top_lab09.v
// Top-level FPGA wrapper for RISC-V Control Path Lab 9
//
// Switch mapping (SW[15:0]):
//   SW[15:9]  = opcode[6:0]   ? instruction bits [6:0]
//   SW[8:6]   = funct3[2:0]   ? instruction bits [14:12]
//   SW[5]     = funct7[5]     ? instruction bit  [30]
//             (only bit of funct7 needed; rest are 0 for RV32I)
//   SW[4:0]   = unused
//
// LED mapping (LED[15:0]):
//   LED[15]   = RegWrite
//   LED[14]   = ALUSrc
//   LED[13]   = MemRead
```

```
//   LED[12]   = MemWrite
//   LED[11]   = MemtoReg
//   LED[10]   = Branch
//   LED[9:8]  = ALUOp[1:0]
//   LED[7:4]  = ALUControl[3:0]
//   LED[3:2]  = FSM state[1:0]
//   LED[1:0]  = unused
//
// FSM states:
//   IDLE     (00) - waiting for button press
//   SAMPLE   (01) - latch switch values
//   DISPLAY  (10) - drive LEDs with control signals
//   WAIT_REL (11) - wait for button release
// ============================================================

module top_lab09 (
   input  wire       clk,
   input  wire       rst,
   input  wire [15:0] sw,   // 16 FPGA slide switches
   input  wire       btn,  // push button (step/sample)
   output wire [15:0] led   // 16 FPGA LEDs
);

   // ---- Debounce the push button ----
   wire btn_clean;
   debouncer u_deb (
      .clk   (clk),
      .pbin  (btn),
      .pbout (btn_clean)
   );

   // ---- Instantiate switches module (Lab 5) ----
   wire [31:0] sw_readData;
   switches u_sw (
      .clk        (clk),
      .rst        (rst),
      .btns       (16'b0),
      .writeData  (32'b0),
      .writeEnable(1'b0),
      .readEnable (1'b1),
      .memAddress (30'b0),
      .switches   (sw),
      .readData   (sw_readData)
   );
```

```
// ---- Extract fields from switches module output ----
// SW[15:9] ? opcode   (instruction bits [6:0])
// SW[8:6]  ? funct3   (instruction bits [14:12])
// SW[5]    ? funct7[5](instruction bit  [30])
wire [6:0] sw_opcode = sw_readData[15:9];
wire [2:0] sw_funct3 = sw_readData[8:6];
wire [6:0] sw_funct7 = {1'b0, sw_readData[5], 5'b0}; // funct7[5] from SW[5]

// ---- Latched instruction fields (set by FSM) ----
reg [6:0] opcode_r;
reg [2:0] funct3_r;
reg [6:0] funct7_r;

// ---- Control signal wires ----
wire      RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch;
wire [1:0] ALUOp;
wire [3:0] ALUControl;

// ---- Instantiate Main Control ----
MainControl u_main (
   .opcode   (opcode_r),
   .RegWrite (RegWrite),
   .ALUSrc   (ALUSrc),
   .MemRead  (MemRead),
   .MemWrite (MemWrite),
   .MemtoReg (MemtoReg),
   .Branch   (Branch),
   .ALUOp    (ALUOp)
);

// ---- Instantiate ALU Control ----
ALUControl u_alu (
   .ALUOp     (ALUOp),
   .funct3    (funct3_r),
   .funct7    (funct7_r),   // full 7 bits [31:25] of instruction
   .ALUControl (ALUControl)
);

// ---- LED data register ----
reg [15:0] led_data;

// ---- Instantiate leds module (Lab 5) ----
leds u_leds (
   .clk        (clk),
   .rst        (rst),
```

```
    .writeData   ({16'b0, led_data}),
    .writeEnable (1'b1),
    .readEnable  (1'b0),
    .memAddress  (30'b0),
    .readData    (),
    .leds        (led)
);

// ---- FSM ----
localparam IDLE     = 2'b00;
localparam SAMPLE   = 2'b01;
localparam DISPLAY  = 2'b10;
localparam WAIT_REL = 2'b11;

reg [1:0] state, next_state;
reg       btn_prev;

// State register
always @(posedge clk or posedge rst) begin
    if (rst) begin
        state    <= IDLE;
        btn_prev <= 1'b0;
    end else begin
        state    <= next_state;
        btn_prev <= btn_clean;
    end
end

// Rising-edge detection
wire btn_rise = btn_clean & ~btn_prev;

// Next-state logic
always @(*) begin
    next_state = state;
    case (state)
        IDLE:     if (btn_rise)   next_state = SAMPLE;
        SAMPLE:                   next_state = DISPLAY;
        DISPLAY:  if (!btn_clean) next_state = WAIT_REL;
        WAIT_REL: if (!btn_clean) next_state = IDLE;
        default:                  next_state = IDLE;
    endcase
end

// Output / datapath logic
always @(posedge clk or posedge rst) begin
```

```
        if (rst) begin
            opcode_r <= 7'b0;
            funct3_r <= 3'b0;
            funct7_r <= 7'b0;
            led_data <= 16'b0;
        end else begin
            case (state)
                SAMPLE: begin
                    opcode_r <= sw_opcode;
                    funct3_r <= sw_funct3;
                    funct7_r <= sw_funct7;
                end
                DISPLAY: begin
                    led_data[15]  <= RegWrite;
                    led_data[14]  <= ALUSrc;
                    led_data[13]  <= MemRead;
                    led_data[12]  <= MemWrite;
                    led_data[11]  <= MemtoReg;
                    led_data[10]  <= Branch;
                    led_data[9:8] <= ALUOp;
                    led_data[7:4] <= ALUControl;
                    led_data[3:2] <= state;
                    led_data[1:0] <= 2'b00;
                end
                default: begin
                    led_data[3:2] <= state;
                end
            endcase
        end
    end

endmodule
```

FSM:

The FSM is implemented directly inside the top module rather than as a separate module. It has 4 states defined using localparams:

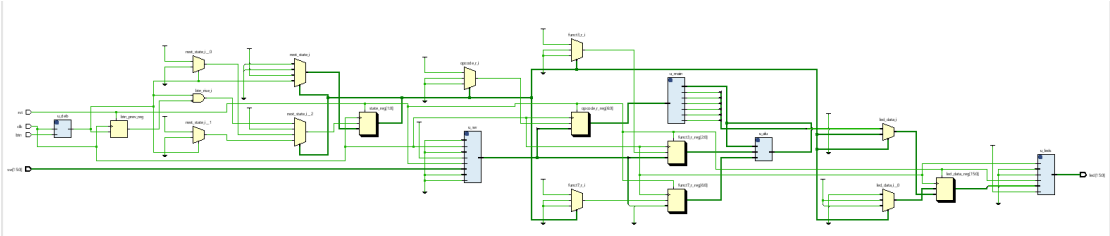
- **IDLE (00):** The FSM sits here waiting for a button press. Rising edge detection (`btn_rise = btn_clean & ~btn_prev`) ensures it only advances once per press
- **SAMPLE (01):** Switch values are latched into registers (opcode_r, funct3_r, funct7_r) on this clock cycle, these are then fed into MainControl and ALUControl
- **DISPLAY (10):** All control signal outputs are mapped to the led_data register which drives the LEDs, showing the result of the current instruction
- **WAIT_REL (11):** The FSM waits here until the button is fully released before returning to IDLE, preventing multiple triggers from a single press

Constraints File:

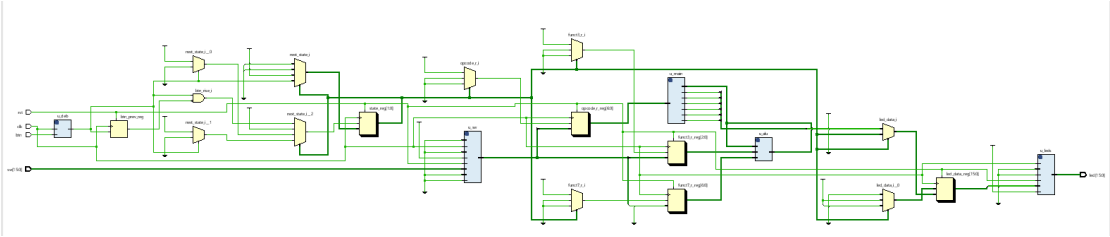
```
## ============================================================
## constraints_basys3.xdc
## Basys3 FPGA pin constraints for Lab 9 - RISC-V Control Path
## ============================================================

## Clock (100 MHz)
set_property PACKAGE_PIN W5      [get_ports clk]
set_property IOSTANDARD  LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

## Reset (BTNC - centre button)
set_property PACKAGE_PIN U18     [get_ports rst]
set_property IOSTANDARD  LVCMOS33 [get_ports rst]

## Step / Sample button (BTNR - right button)
set_property PACKAGE_PIN T17     [get_ports btn]
set_property IOSTANDARD  LVCMOS33 [get_ports btn]

## ---- Slide Switches ----
## SW[15] = opcode[6]   SW[14] = opcode[5]   SW[13] = opcode[4]
## SW[12] = opcode[3]   SW[11] = opcode[2]   SW[10] = opcode[1]
## SW[9]  = opcode[0]
## SW[8]  = funct3[2]   SW[7]  = funct3[1]   SW[6]  = funct3[0]
## SW[5]  = funct7 (bit 30)
## SW[4:0] unused
```

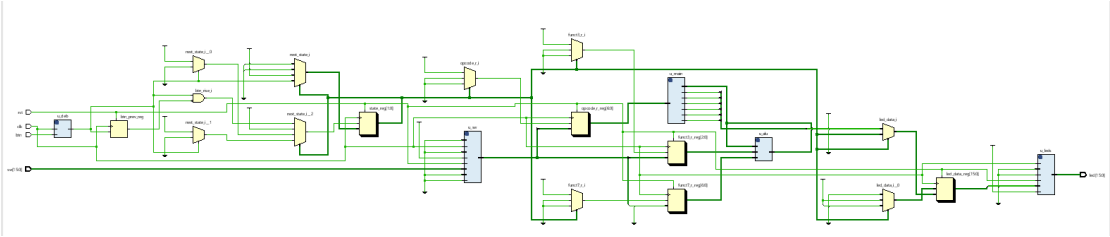
```
set_property PACKAGE_PIN V17  [get_ports {sw[0]}]
set_property PACKAGE_PIN V16  [get_ports {sw[1]}]
set_property PACKAGE_PIN W16  [get_ports {sw[2]}]
set_property PACKAGE_PIN W17  [get_ports {sw[3]}]
set_property PACKAGE_PIN W15  [get_ports {sw[4]}]
set_property PACKAGE_PIN V15  [get_ports {sw[5]}]
set_property PACKAGE_PIN W14  [get_ports {sw[6]}]
set_property PACKAGE_PIN W13  [get_ports {sw[7]}]
set_property PACKAGE_PIN V2   [get_ports {sw[8]}]
set_property PACKAGE_PIN T3   [get_ports {sw[9]}]
set_property PACKAGE_PIN T2   [get_ports {sw[10]}]
set_property PACKAGE_PIN R3   [get_ports {sw[11]}]
set_property PACKAGE_PIN W2   [get_ports {sw[12]}]
set_property PACKAGE_PIN U1   [get_ports {sw[13]}]
set_property PACKAGE_PIN T1   [get_ports {sw[14]}]
set_property PACKAGE_PIN R2   [get_ports {sw[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[*]}]

## ---- LEDs ----
## LED[15] = RegWrite    LED[14] = ALUSrc
## LED[13] = MemRead     LED[12] = MemWrite
## LED[11] = MemtoReg    LED[10] = Branch
## LED[9:8]= ALUOp       LED[7:4]= ALUControl
## LED[3:2]= FSM state   LED[1:0]= unused

set_property PACKAGE_PIN U16  [get_ports {led[0]}]
set_property PACKAGE_PIN E19  [get_ports {led[1]}]
set_property PACKAGE_PIN U19  [get_ports {led[2]}]
set_property PACKAGE_PIN V19  [get_ports {led[3]}]
set_property PACKAGE_PIN W18  [get_ports {led[4]}]
set_property PACKAGE_PIN U15  [get_ports {led[5]}]
set_property PACKAGE_PIN U14  [get_ports {led[6]}]
set_property PACKAGE_PIN V14  [get_ports {led[7]}]
set_property PACKAGE_PIN V13  [get_ports {led[8]}]
set_property PACKAGE_PIN V3   [get_ports {led[9]}]
set_property PACKAGE_PIN W3   [get_ports {led[10]}]
set_property PACKAGE_PIN U3   [get_ports {led[11]}]
set_property PACKAGE_PIN P3   [get_ports {led[12]}]
set_property PACKAGE_PIN N3   [get_ports {led[13]}]
set_property PACKAGE_PIN P1   [get_ports {led[14]}]
set_property PACKAGE_PIN L1   [get_ports {led[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[*]}]
```

RTL Schematic:
(Pdf also uploaded on github)



The schematic shows the synthesized RTL view of the full top_lab09 module. The signal flow goes left to right — inputs (clk, rst, btn, sw) on the left and LED outputs on the right
On the far left, the debouncer block (u_deb) processes the raw button input and produces btn_clean. An AND gate immediately after implements the rising edge detection (`btn_rise = btn_clean & ~btn_prev`) using the btn_prev_reg flip flop
The switches module (u_sw) reads SW[15:0] and outputs sw_readData[31:0]. Bits [15:9] are routed as opcode, [8:6] as funct3, and [5] as funct7[5]
The FSM state register (state_reg[1:0]) is a 2-bit async reset flip flop. The next-state logic is implemented as a series of RTL_MUX blocks which Vivado synthesized from the case statement in the Verilog code
In the SAMPLE state, the MUX blocks (opcode_r_i, funct3_r_i, funct7_r_i) select the switch values and latch them into their respective registers (opcode_r_reg, funct3_r_reg, funct7_r_reg)
The latched values feed into MainControl (u_main) which produces all 7 control signals, ALUOp then connects directly into ALUControl (u_alu) which produces the 4-bit ALUControl output
In the DISPLAY state, the led_data MUX (led_data_i) selects the control signal values and latches them into led_data_reg[15:0], which is then passed to the leds module (u_leds) to drive the physical LEDs
All registers in the schematic are asynchronous reset flip flops (RTL_REG_ASYNC) which matches the `always @(posedge clk or posedge rst)` blocks in the Verilog code





nc

Lab 9: Design and FPGA Implementation of the Control Path for a Single-Cycle RISC-V Processor

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/20	/20
Task 2	/20	/15		
Task 3	/10	/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	**Program/Code / Simulation Model/ Network Model**	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.

LR5	**Results & Plots**	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	**Report**	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.